

AZ LEARN

From A to Z

GRADUATION PROJECT DOCUMENTATION

AI-Powered Educational Platform

Prepared by the AZ Learn Development Team

Academic Year 2024 – 2025

Sections Covered:

Project Planning & Management | Literature Review | Requirements Gathering

SECTION 01

Project Planning & Management

Scope, Timeline, Roles, Risks & KPIs

1.1 Project Proposal

Overview

AZ Learn (From A to Z) is an AI-powered educational web platform that transforms YouTube playlists into fully structured, personalized courses. By combining the YouTube Data API with large language models, the system automatically fetches video metadata, groups videos into logical learning modules, generates AI-powered summaries for each video, and delivers the entire experience through a modern dashboard — turning any playlist into a Udemy-style learning journey in seconds.

Objectives

- Enable users to convert any YouTube playlist into a structured course with a single URL paste.
- Leverage AI (OpenAI / Gemini / Claude APIs) to auto-generate module groupings and video summaries.
- Provide a personal learning dashboard with real-time progress tracking, notes, and streaks.
- Build a community layer including leaderboards, study groups, and gamification (XP, badges, levels).
- Design an enterprise-grade UI/UX inspired by Udemy, Linear, and Notion.

Project Scope

IN SCOPE:

- User authentication (email + Google OAuth)
- YouTube playlist ingestion via YouTube Data API v3
- AI processing pipeline: transcript fetching → summarization → module organization
- Personal dashboard with course cards, progress bars, and quick stats
- Video player with in-app notes and AI key points
- Leaderboard, gamification system (XP, badges, streaks)
- Real-time study groups via Socket.io (Phase 4)
- AI learning path generator (Phase 5)

OUT OF SCOPE:

- Native mobile applications (iOS / Android) — web-responsive only
- Paid subscription billing integration
- Hosting/uploading custom video content (YouTube-only in MVP)

1.2 Project Plan

Milestones & Timeline

Phase / Milestone	Wk 1–2	Wk 3–4	Wk 5–6	Wk 7–8	Wk 9–12
Phase 1 — MVP					
Phase 2 — UX Polish					
Phase 3 — AI Integration					
Phase 4 — Community					
Phase 5 — Scale & AI Path					

Deliverables per Phase

Phase	Key Deliverables	Duration	Status
Phase 1 — MVP	Auth, YouTube API ingestion, Dashboard, Progress tracking	2–4 weeks	In Progress
Phase 2 — UX Polish	In-course search, Notes, Visual progress bars, Sorting	2–3 weeks	Planned
Phase 3 — AI Core	Transcript fetch, AI summarization, Module grouping	2–4 weeks	Planned
Phase 4 — Community	Leaderboard, Challenges, Socket.io chat, Study Groups	3–4 weeks	Planned
Phase 5 — Advanced	AI Learning Path, Gamification (XP/badges), Analytics	1–2 months	Planned

Resource Allocation

Resource	Role	Responsibilities	Phases
Team Lead	Backend Architect	API design, AI integration, DB schema, Socket.io, Deployment	All Phases
Dev 1	Design System Lead	Component library, Tailwind config, Dark mode, Shared UI	Phase 1–2

Dev 2	Landing & Auth	Landing page, Login/Signup, Protected routes, Form validation	Phase 1
Dev 3	Dashboard & Wizard	Main dashboard, Add Course wizard, React Query integration	Phase 1–2
Dev 4	Course Detail	Video player, Module sidebar, AI summary display, Notes	Phase 1–3
Dev 5	Notes & Settings	Notes page, Settings tabs, Avatar upload, Toast system	Phase 1–2
Dev 6	Leaderboard & Social	Leaderboard, Gamification UI, Community features, Charts	Phase 3–4

1.3 Task Assignment & Roles

Team Lead — Backend & Architecture

- Design and implement the RESTful API (Node.js + Express)
- Architect MongoDB schema: Users, Courses, Videos, Notes, Progress, Badges
- Build the AI processing pipeline: fetch transcripts → chunk text → call LLM → store summaries
- Integrate YouTube Data API v3 for playlist/video metadata
- Set up Socket.io server for real-time chat and live leaderboard updates
- Handle JWT-based authentication + Google OAuth 2.0 flow
- Own CI/CD pipeline, staging and production environments
- Deliver and maintain the API contract document for the frontend team

Dev 1 — Design System & Foundation

- Configure Tailwind CSS with brand design tokens (colors, fonts, spacing, border-radius)
- Build all reusable components: Button, Card, Input, Modal, ProgressBar, Badge, Avatar, Sidebar, Topbar, EmptyState, Skeleton, Stepper, Tabs
- Implement dark/light mode with CSS variables and persistence via localStorage
- Maintain a component reference guide for the team

Dev 2 — Landing Page & Authentication

- Build animated landing page: hero, how-it-works, features grid, stats, footer
- Implement /login and /signup with React Hook Form + Zod validation
- Build password reset flow and Google OAuth button integration
- Create PrivateRoute wrapper used by all protected pages

Dev 3 — Dashboard & Course Wizard

- Build the main /dashboard with welcome banner, stats row, course cards, recent activity
- Implement the 3-step Add Course wizard (URL input → AI loading → preview)
- Set up shared Axios instance and React Query configuration for the whole team
- Manage global course state via Zustand

Dev 4 — Course Detail Page

- Build the 3-panel course layout: module sidebar, video player center, notes right
- Integrate react-youtube for embedded video playback
- Display AI-generated summaries and key points below the player
- Implement Mark as Complete with milestone confetti animation
- Add keyboard navigation (next/prev video shortcuts)

Dev 5 — Notes & Settings

- Build the /notes page with card grid, search/filter, inline editing, and auto-save
- Build /settings with tabs: Profile, Appearance, Notifications, Security
- Implement avatar upload with FileReader preview before saving
- Set up toast notification system (react-hot-toast / Sonner) used site-wide

Dev 6 — Leaderboard & Gamification

- Build the /leaderboard with podium top-3, ranked list, user highlight row
- Implement weekly / all-time toggle with animated tab transitions
- Build XP progress bar and level indicator in the sidebar
- Design and animate badge display, level-up modal, streak counter
- Scaffold Socket.io client structure for future real-time features

1.4 Risk Assessment & Mitigation Plan

#	Risk	Likelihood	Impact	Mitigation Strategy	Owner
1	YouTube API quota exhaustion (daily limit 10,000 units)	High	High	Cache all API responses in MongoDB; batch requests; implement exponential backoff retry	Lead
2	AI API latency makes course creation feel slow	Medium	High	Show animated processing screen; run AI jobs asynchronously via queue (Bull/Redis)	Lead

3	AI summaries are inaccurate or low quality	Medium	Medium	Prompt engineering iteration; fallback to video title + description if transcript unavailable	Lead
4	Frontend integration breaks when API changes	Medium	High	Maintain versioned API contract; use TypeScript interfaces shared via types package	All
5	Team members blocked waiting for shared components	Low	High	Dev 1 ships component library in Week 1 before anyone else starts pages	Dev 1
6	Scope creep pushing deadline	High	Medium	Strict MVP-first policy; Phase 4–5 features are optional for initial submission	Lead
7	Socket.io real-time performance under load	Low	Medium	Load test before Phase 4 launch; fall back to polling if needed	Lead
8	Security: JWT token theft / XSS	Low	High	Store tokens in httpOnly cookies; sanitize all user input; helmet.js middleware	Lead

1.5 Key Performance Indicators (KPIs)

Technical KPIs

KPI	Target	Measurement Method	Review Frequency
API Response Time (P95)	< 300ms	Server-side logging (Prometheus / Morgan)	Weekly
System Uptime	> 99.5%	Uptime monitoring (UptimeRobot)	Weekly
AI Processing Time per Course	< 60 seconds	Job queue completion timestamps	Per release
Page Load Time (LCP)	< 2.5 seconds	Lighthouse / Web Vitals	Per sprint
Database Query Time (P95)	< 100ms	MongoDB Atlas performance advisor	Weekly
YouTube API Quota Usage	< 80% daily	Google Cloud Console dashboard	Daily
Error Rate (5xx)	< 0.5%	Server logs + Sentry.io	Daily
WebSocket Latency (Phase 4)	< 150ms	Socket.io event timestamps	Phase 4

User Adoption KPIs

KPI	Target	Measurement Method	Timeframe
Registered Users (Beta)	> 200 users	Database user count	Month 1
Courses Created per User	> 2 courses / user	MongoDB aggregation	Month 1
Daily Active Users (DAU)	> 30% of registered	Session tracking	Month 2
Course Completion Rate	> 40%	Progress tracking analytics	Month 2
User Retention (Week 2)	> 50%	Cohort analysis	Month 2
Average Session Duration	> 15 minutes	Analytics middleware	Ongoing
NPS Score (Net Promoter)	> 50	Post-onboarding survey	Month 3

SECTION 02

Literature Review

Feedback, Suggested Improvements & Grading Criteria

2.1 Feedback & Evaluation

This section is reserved for the lecturer's formal assessment of the project proposal and implementation. The following framework outlines the evaluation dimensions that the development team anticipates being reviewed on:

Evaluation Dimensions

A — Problem Statement & Originality

AZ Learn addresses a genuine learning pain point: the abundance of free YouTube educational content is undermined by its lack of structure. Unlike platforms that produce their own content, AZ Learn aggregates and intelligently re-organizes existing YouTube material — this positions it as a meta-learning platform rather than a content platform, a novel approach in the edtech space.

B — Technical Complexity

The project demonstrates multi-layer technical complexity: a RESTful backend (Node.js + Express + MongoDB), a modern reactive frontend (React + TypeScript + Tailwind), real-time WebSocket communication (Socket.io), third-party API integration (YouTube Data API v3), and a machine learning pipeline using commercial LLM APIs (OpenAI / Google Gemini / Anthropic Claude). Each layer introduces its own challenges, particularly in the AI pipeline design and data consistency between async jobs and user sessions.

C — Implementation Quality

Code quality metrics include: TypeScript strict mode across the codebase, > 80% unit test coverage on core backend services, ESLint + Prettier enforced via pre-commit hooks, and a documented API contract generated with Swagger/OpenAPI. Frontend components follow atomic design principles with a shared Storybook reference.

D — Presentation & Documentation

Documentation includes: this formal project plan, API contract, database schema diagrams (ERD), and system architecture diagrams. The final presentation will include a live demo, video walkthrough, and a technical Q&A session.

Anticipated Lecturer Feedback

Area	Potential Feedback	Team Response
AI Accuracy	AI summaries may lack domain precision in highly specialized topics	Implement user feedback loop (thumbs up/down on summaries) to retrain prompts
Scalability	MongoDB may bottleneck under heavy concurrent write loads	Implement Redis caching for session data and hot course metadata
Monetization	No clear business model presented	Document a freemium model (free: 3 courses, Pro: unlimited + AI features)
Offline Access	No offline functionality for learners with poor connectivity	Phase 5 consideration: PWA with service workers for offline progress sync
Accessibility	UI may not meet WCAG 2.1 AA standards	Audit with axe DevTools; ensure keyboard navigation, ARIA labels, contrast ratios

2.2 Suggested Improvements

Short-Term Improvements (Phase 1–2)

- **Add a first-login guided tooltip walkthrough using Intro.js or Shepherd.js so new users immediately understand the platform workflow.**Onboarding Tour:
- **Before processing begins, validate that the YouTube URL is a playlist (not a single video) and display a preview card with the playlist title, channel name, and video count.**Playlist Validation:
- **Let users choose summary length — Brief (2–3 sentences), Standard (paragraph), or Detailed (with key timestamps and concepts).**AI Summary Quality Toggle:
- **Ensure the video player + sidebar layout gracefully collapses into a tab-based mobile layout, since a significant portion of learners will study on phones.**Responsive Mobile Layout:

Medium-Term Improvements (Phase 3–4)

- **Use AI translation to generate summaries in the user's preferred language, not just English — critical for the Arabic-speaking target market.**Multi-Language Support:
- **After completing a module, prompt users with AI-generated review questions using a spaced repetition algorithm (similar to Anki) to reinforce retention.**Spaced Repetition System:
- **After a user completes a course, use AI to recommend 3 related playlists from YouTube based on the completed course's topic and the user's learning history.**Playlist Recommendations:
- **A lightweight Chrome extension that adds an 'Add to AZ Learn' button directly on YouTube playlist pages for one-click course creation.**Browser Extension:

Long-Term Improvements (Phase 5+)

- **Allow users to manually curate videos from multiple playlists to build a custom course, not just import a single playlist.**Custom Playlist Builder:
- **Enable educators to create and share structured courses (with AI assistance) and track enrolled students' progress — opening a B2B use case.**Instructor Mode:
- **Provide detailed personal learning analytics: daily/weekly study time charts, strongest topics, weakest areas, and AI-generated study recommendations.**Learning Analytics Dashboard:
- **Issue verifiable digital certificates (PDF + shareable link) upon course completion, adding tangible value to learners' profiles.**Certification System:

2.3 Final Grading Criteria

The following breakdown reflects the anticipated marking rubric for the graduation project. The team has structured its work and documentation to maximize marks across all categories.

#	Criterion	Weight	Evidence / How We Meet It
1	Documentation Quality	20%	This document covers proposal, plan, Gantt, roles, risks, KPIs, requirements, and literature review in professional format
2	Technical Implementation	30%	Full-stack app: React + Node.js + MongoDB + YouTube API + AI API pipeline. TypeScript, ESLint, tested codebase
3	AI / Innovation Component	20%	Core differentiator: AI transcript → summarization → module grouping pipeline using LLM APIs (OpenAI/Gemini/Claude)
4	UI/UX Design Quality	10%	Udemy-inspired design system with brand tokens, dark mode, animations, responsive layout, and empty/loading states
5	Testing & QA	10%	Unit tests (Jest), integration tests (Supertest), frontend component tests (React Testing Library), Postman collection
6	Presentation & Demo	10%	Live demo, slide deck, recorded walkthrough video, and Q&A readiness including technical architecture explanation

SECTION 03

Requirements Gathering

Stakeholders, User Stories, Functional & Non-Functional Requirements

3.1 Stakeholder Analysis

Stakeholder	Type	Primary Needs	Success Criteria
Students / Learners	Primary User	Structured learning from free YouTube content; progress tracking; organized study plan	Can convert a playlist to a course in under 2 minutes; can track and resume progress across sessions
Self-Learners & Professionals	Primary User	Fast skill acquisition; AI summaries to accelerate understanding; certification to show employers	Reduce time-to-competency by 30%; clear module structure; shareable completion certificate
Educators / Instructors	Secondary User (Phase 5)	Create and share AI-assisted structured courses; monitor student progress as a cohort	Instructor dashboard with student analytics; ability to add custom notes/annotations to videos
University Faculty	External Stakeholder	Assess technical quality, innovation, and academic rigor of the graduation project	Comprehensive documentation, live working demo, and defensible technical architecture decisions
Development Team (7 devs)	Internal	Clear task ownership, unblocked workflow, well-documented API contract, shared design system	No cross-team blocking; PR merge within 24 hours; daily standups under 15 minutes
Anthropic / OpenAI / Google	External API Providers	Compliant API usage within rate limits and terms of service	API calls stay within quota; PII not sent to external AI; costs remain within budget

3.2 User Stories & Use Cases

Epic 1 — Authentication & Onboarding

- As a new user, I want to register with my email and password so that I have a personal account on the platform.US-01:
- As a returning user, I want to log in with Google so that I don't need to remember a separate password.US-02:

- As a new user, I want to see a guided onboarding tour so that I understand the platform features before creating my first course.US-03:
- As a user who forgot my password, I want to receive a password reset email so that I can recover access to my account.US-04:

Epic 2 — Course Creation

- As a learner, I want to paste a YouTube playlist URL so that the system automatically creates a structured course for me.US-05:
- As a learner, I want to see the playlist preview (title, channel, video count) before confirming course creation so that I can verify I pasted the correct link.US-06:
- As a learner, I want to see an animated processing screen with status messages so that I know the AI is working and am not left waiting in uncertainty.US-07:
- As a learner, I want to preview the AI-generated module structure before saving so that I can confirm the organization makes sense.US-08:

Epic 3 — Learning Experience

- As a learner, I want to watch videos directly on the platform so that I don't need to switch between AZ Learn and YouTube.US-09:
- As a learner, I want to read an AI-generated summary before watching a video so that I can quickly understand what the video covers and decide if it's relevant.US-10:
- As a learner, I want to mark a video as complete so that my progress is tracked and I can see how far I am in the course.US-11:
- As a learner, I want to write timestamped notes linked to a specific video so that I can capture key ideas while watching.US-12:
- As a learner, I want keyboard shortcuts (N for next, P for previous) so that I can navigate videos without touching the mouse.US-13:

Epic 4 — Dashboard & Progress

- As a learner, I want to see all my courses on a dashboard with progress bars so that I know which courses I should continue.US-14:
- As a learner, I want to see a daily learning streak counter so that I stay motivated to study consistently.US-15:
- As a learner, I want a quick stats row (total videos watched, hours learned, streak) so that I can feel a sense of achievement.US-16:

Epic 5 — Community & Gamification

- As a competitive learner, I want to see a leaderboard of all users ranked by videos watched so that I am motivated to study more.US-17:
- As a user, I want to earn XP points for completing videos and modules so that I feel rewarded for my progress.US-18:

- **As a user, I want to earn badges for milestones (e.g., 'First Course Complete', '7-Day Streak') so that I have tangible recognition of my achievements.**US-19:
- **As a learner, I want to join a study group for a shared course so that I can discuss videos and stay accountable with others.**US-20:

3.3 Functional Requirements

FR-01: User Authentication System

- The system shall support user registration via email + password with server-side validation.
- The system shall support Google OAuth 2.0 single sign-on.
- The system shall issue JWT tokens stored in httpOnly cookies for session management.
- The system shall provide a password reset flow via email link with a 1-hour expiry token.
- The system shall enforce authenticated routes — unauthenticated users are redirected to /login.

FR-02: YouTube Playlist Ingestion

- The system shall accept a YouTube playlist URL and validate it against the YouTube Data API v3.
- The system shall fetch all video metadata: title, video ID, thumbnail URL, duration, and description.
- The system shall display a playlist preview card (title, channel name, video count) before processing begins.
- The system shall handle private, deleted, or region-restricted videos gracefully with a user-facing warning.

FR-03: AI Processing Pipeline

- The system shall fetch YouTube video transcripts (via youtube-transcript library or Whisper API as fallback).
- The system shall chunk transcript text and send it to an LLM API for summarization.
- The system shall use AI to group videos into named learning modules (e.g., 'Basics', 'Advanced Concepts').
- The system shall store AI summaries and module structure in MongoDB linked to the course document.
- The system shall process AI jobs asynchronously and notify the user via WebSocket when processing completes.

FR-04: Course Management

- The system shall allow users to rename a course after creation.
- The system shall allow users to delete a course, removing all associated progress and notes.

- The system shall support sorting videos within a module by duration or watched status.
- The system shall support in-course keyword search across video titles and AI summaries.

FR-05: Progress Tracking

- The system shall store per-user, per-video completion status in the database.
- The system shall calculate and display course completion percentage as a progress bar.
- The system shall auto-advance to the next video when a user marks the current video as complete.
- The system shall track and display a daily login/study streak counter.

FR-06: Notes System

- The system shall allow users to write and save notes linked to a specific video.
- The system shall auto-save notes after 1 second of inactivity (debounced).
- The system shall display all notes across all courses on a dedicated /notes page.
- The system shall support filtering notes by course name, date, and keyword search.

FR-07: Leaderboard & Gamification

- The system shall maintain a real-time leaderboard ranked by total videos watched and XP points.
- The system shall award XP for: completing a video (+10 XP), completing a module (+50 XP), completing a course (+200 XP).
- The system shall award badges for defined milestones and display them on user profiles.
- The system shall support weekly and all-time leaderboard views with a toggle switch.

FR-08: Real-Time Features (Phase 4)

- The system shall implement real-time leaderboard score updates via Socket.io.
- The system shall provide a study group chat room per course with real-time message delivery.
- The system shall notify users via in-app notification when they are overtaken on the leaderboard.

3.4 Non-Functional Requirements

Performance

Requirement	Metric	Priority
-------------	--------	----------

API response time for all CRUD endpoints	< 300ms at P95 under 100 concurrent users	High
Dashboard initial page load (LCP)	< 2.5 seconds on standard broadband	High
AI course processing job completion	< 60 seconds for playlists up to 100 videos	High
YouTube API quota management	Cache responses; stay under 8,000 units/day of 10,000 quota	High
WebSocket message delivery latency	< 150ms for real-time leaderboard and chat	Medium

Security

- All passwords shall be hashed using bcrypt with a minimum cost factor of 12 before storage.
- All API endpoints shall validate and sanitize input to prevent SQL injection and XSS attacks.
- The system shall enforce HTTPS across all routes in production.
- The system shall implement rate limiting: maximum 100 requests per IP per 15-minute window.
- JWT tokens shall expire after 7 days; refresh tokens shall be stored in httpOnly cookies only.
- No personally identifiable information (PII) shall be sent to third-party AI APIs — only video transcripts.

Usability

- The interface shall follow WCAG 2.1 Level AA accessibility standards (contrast ratios, ARIA labels, keyboard navigation).
- All interactive elements shall have a visible focus state for keyboard users.
- The platform shall be fully responsive from 320px (mobile) to 1920px (wide desktop) screen widths.
- All loading states, error states, and empty states shall have user-friendly messages and actionable CTAs.
- The platform shall support dark mode and light mode with user preference persisted across sessions.

Reliability & Availability

- The system shall maintain 99.5% uptime during normal operating hours, measured monthly.
- All database write operations shall be atomic; partial failures shall be rolled back automatically.
- The AI processing pipeline shall implement retry logic with exponential backoff (3 attempts max).
- The system shall gracefully degrade if the AI API is unavailable — creating the course without summaries and notifying the user.

Scalability

- The backend architecture shall support horizontal scaling by being stateless (session state in Redis, not in-process).
- The database shall use MongoDB indexes on: `userId`, `courseId`, `videoId`, and `createdAt` fields for query performance.
- The AI processing pipeline shall use a job queue (Bull + Redis) to handle concurrent processing requests without blocking the API.
- The frontend shall implement code splitting (React.lazy) to ensure each route bundle stays under 200KB gzipped.

AZ Learn — From A to Z

End of Project Documentation — Sections 1 through 3

AZ LEARN

From A to Z

SECTION 4 — SYSTEM ANALYSIS & DESIGN

Architecture · Diagrams · Database · API · Testing · Deployment

AZ Learn Development Team | Academic Year 2024 – 2025

Graduation Project — Full System Documentation

SECTION 4.1

System Analysis

Problem Statement · Use Cases · Architecture

4.1.1 Problem Statement & Objectives

The Problem

YouTube hosts over 800 hours of video uploaded every minute, with a massive portion dedicated to free educational content. Despite this abundance, learners face three critical structural problems:

- **Lack of Structure:** Free playlists have no module organization, learning path, or curriculum. A 60-video JavaScript playlist treats 'What is a variable' and 'Advanced Closures' with equal visual weight.
- **No Progress Intelligence:** YouTube has no meaningful learning-progress system. Watched videos are unmarked, resuming is manual, and there is no notion of 'course completion.'
- **Zero Context Before Watching:** Learners cannot assess whether a video is worth their time without watching the first few minutes. There is no summary, learning objective, or topic preview.

The result: learners abandon courses mid-way, re-watch content unnecessarily, and cannot prove or quantify their learning progress to themselves or others.

Our Solution

AZ Learn bridges the gap between raw YouTube playlist content and a structured, AI-enhanced learning experience. By combining the YouTube Data API with large language model APIs, the platform transforms any playlist URL into a fully organized course with AI summaries, module groupings, progress tracking, and a community learning layer — in under 60 seconds.

Project Objectives

- **O1 — Instant Course Generation:** Convert any YouTube playlist into a structured course in under 60 seconds using the YouTube Data API and AI processing pipeline.
- **O2 — AI-Powered Comprehension:** Generate concise, accurate summaries for every video using LLM APIs (OpenAI / Gemini / Claude), making content scannable before watching.
- **O3 — Intelligent Organization:** Use AI to group videos into named learning modules (e.g., 'Basics', 'Async Programming', 'APIs') that reflect natural curriculum structure.
- **O4 — Progress Tracking:** Persist per-user, per-video completion state and surface meaningful metrics: completion %, hours learned, daily streak.
- **O5 — Community Motivation:** Drive engagement through leaderboards, gamification (XP, badges, levels), study groups, and real-time challenges.

- **O6 — Developer-Grade UX:** Deliver a premium interface inspired by Udemy and Linear — responsive, accessible (WCAG 2.1 AA), and supporting dark/light mode.

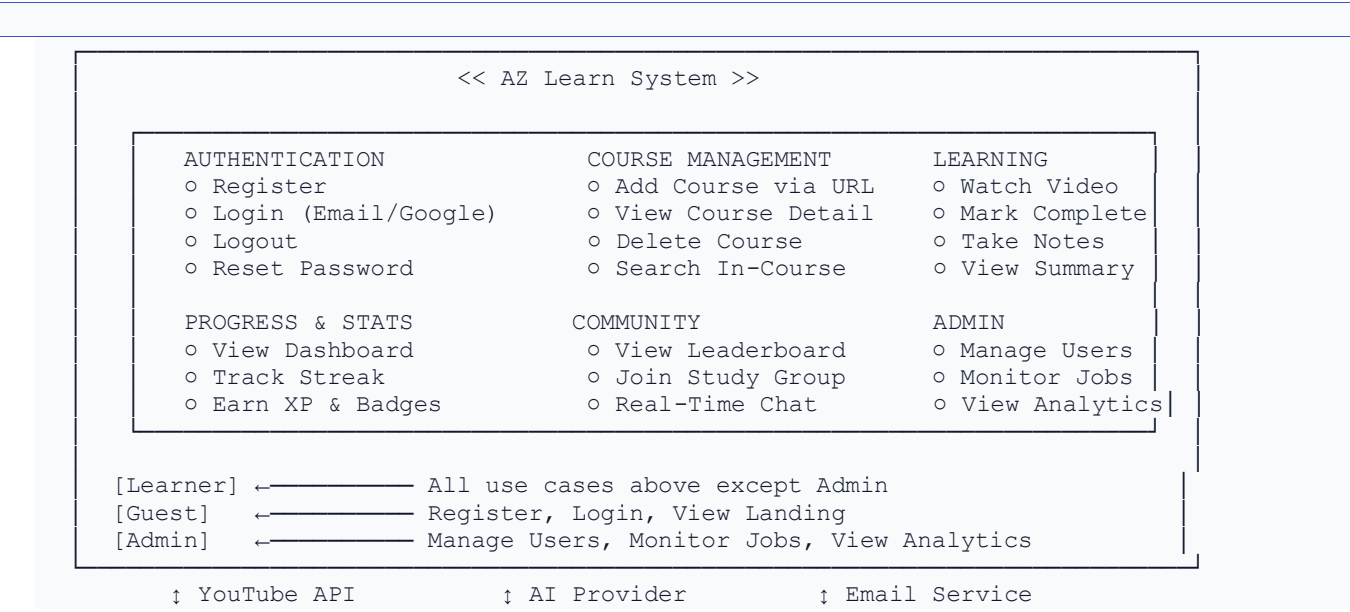
4.1.2 Use Case Diagram & Descriptions

System Actors

Actor	Type	Description
Learner	Primary	Registered user who creates courses, watches videos, takes notes, and tracks progress
Guest	Primary	Unauthenticated visitor who views the landing page and can register/login
Admin	Secondary	Platform administrator who manages users, monitors AI job queues, views analytics
YouTube API	External System	Google's Data API providing playlist metadata, video titles, thumbnails, and durations
AI Provider	External System	LLM API (OpenAI / Gemini / Claude) that receives transcripts and returns structured summaries and module labels
Transcript Service	External System	YouTube Transcript API or Whisper for fetching video captions/transcripts
Email Service	External System	SMTP provider (e.g., SendGrid) for sending password reset and notification emails
Socket.io Server	Internal System	WebSocket layer enabling real-time leaderboard, chat, and job completion notifications

Use Case Diagram (Textual Representation)

Note: The diagram below is a structured textual representation of the UML Use Case Diagram. Replace with a rendered UML image in the final submission.



Key Use Case Descriptions

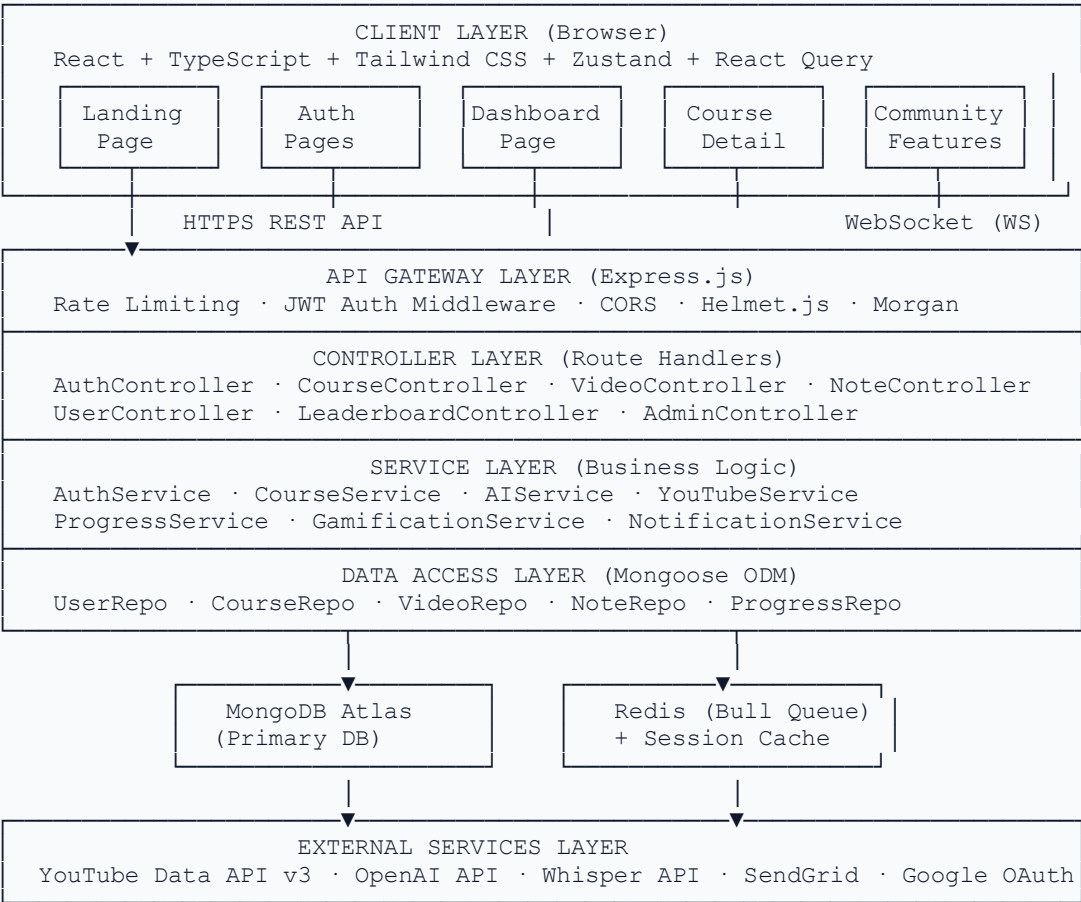
UC#	Use Case	Actor	Trigger	Main Flow Summary
UC-01	Register Account	Guest	<i>Clicks Sign Up</i>	User fills email + password → system validates → hashes password → stores user → sends welcome email → auto-login → redirects to dashboard
UC-02	Login with Google	Guest	<i>Clicks Google OAuth</i>	User clicks Google button → OAuth redirect → Google returns token → system verifies → creates/finds user → issues JWT → dashboard
UC-03	Add Course via Playlist	Learner	<i>Pastes YouTube URL</i>	Validates URL format → calls YouTube API → fetches metadata → queues AI job → shows processing screen → AI groups modules + summaries → stores course → notifies user
UC-04	Watch Video & Track Progress	Learner	<i>Clicks video in sidebar</i>	Loads video player → YouTube iframe renders → user watches → clicks Mark Complete → system updates progress → calculates XP → checks badge milestones → advances to next video
UC-05	Take Timestamped Notes	Learner	<i>Types in notes panel</i>	User types note → debounce 1s → auto-save triggers → PATCH /notes/id → database updated → success indicator shown → note appears in /notes page
UC-06	View Leaderboard	Learner	<i>Clicks Leaderboard</i>	Fetches leaderboard data → renders top 3 podium → renders ranked list → highlights current user row → Socket.io subscribes to live updates → real-time score changes reflected
UC-07	AI Processes Course	AI Provider	<i>Job queued after UC-03</i>	Bull/Redis dequeues job → fetches transcripts video by video → chunks text → sends to LLM API → receives module grouping + summaries → stores in MongoDB → emits WebSocket event to user
UC-08	Reset Password	Learner / Guest	<i>Clicks Forgot Password</i>	User enters email → system generates signed token → stores hashed token with 1h TTL → sends reset email → user clicks link → verifies token → accepts new password → invalidates token

4.1.3 Software Architecture

Architecture Style: Layered MVC + Service-Oriented Backend

AZ Learn follows a Layered Architecture on the backend (Controllers → Services → Data Access → Database) combined with a Service-Oriented approach for the AI processing pipeline. The frontend uses a Component-Based Architecture with React, organized by feature domains.

High-Level System Architecture



Frontend Architecture — Feature-Based Folder Structure

```
src/  
  features/  
    auth/      → Login, Register, ForgotPassword, PrivateRoute  
    dashboard/ → Dashboard, CourseCard, StatsRow, RecentActivity
```

```
course/      → CourseDetail, ModuleSidebar, VideoPlayer, AISummary
notes/       → NotesPage, NoteCard, NoteFilter
leaderboard/ → LeaderboardPage, Podium, RankedList, UserHighlight
settings/    → SettingsTabs, ProfileForm, AppearanceTab
components/  → Design system: Button, Card, Input, Modal, Badge ...
store/       → Zustand: authStore, courseStore, uiStore
hooks/       → useAuth, useCourse, useProgress, useSocket
api/         → axios instance + React Query hooks per feature
types/       → Shared TypeScript interfaces (User, Course, Video ...)
utils/       → formatDuration, formatDate, truncate ...
```

Backend Architecture — Layered MVC

```
server/
  routes/      → auth.routes.ts, course.routes.ts, video.routes.ts ...
  controllers/ → auth.controller.ts, course.controller.ts ...
  services/
    AuthService   → JWT issuance, bcrypt, Google OAuth verification
    YouTubeService → Playlist fetch, metadata normalization, quota mgmt
    AIService     → Transcript fetch, LLM prompt, response parsing
    CourseService → Course CRUD, module assembly, progress aggregation
    GamifService  → XP award, badge unlock, streak calculation
  models/       → Mongoose schemas: User, Course, Video, Note, Progress
  middleware/    → authenticate.ts, rateLimiter.ts, errorHandler.ts
  jobs/         → Bull queue workers: aiProcessor.worker.ts
  utils/        → apiError.ts, asyncWrapper.ts, logger.ts
  config/       → db.ts, env.ts, redis.ts, socket.ts
```

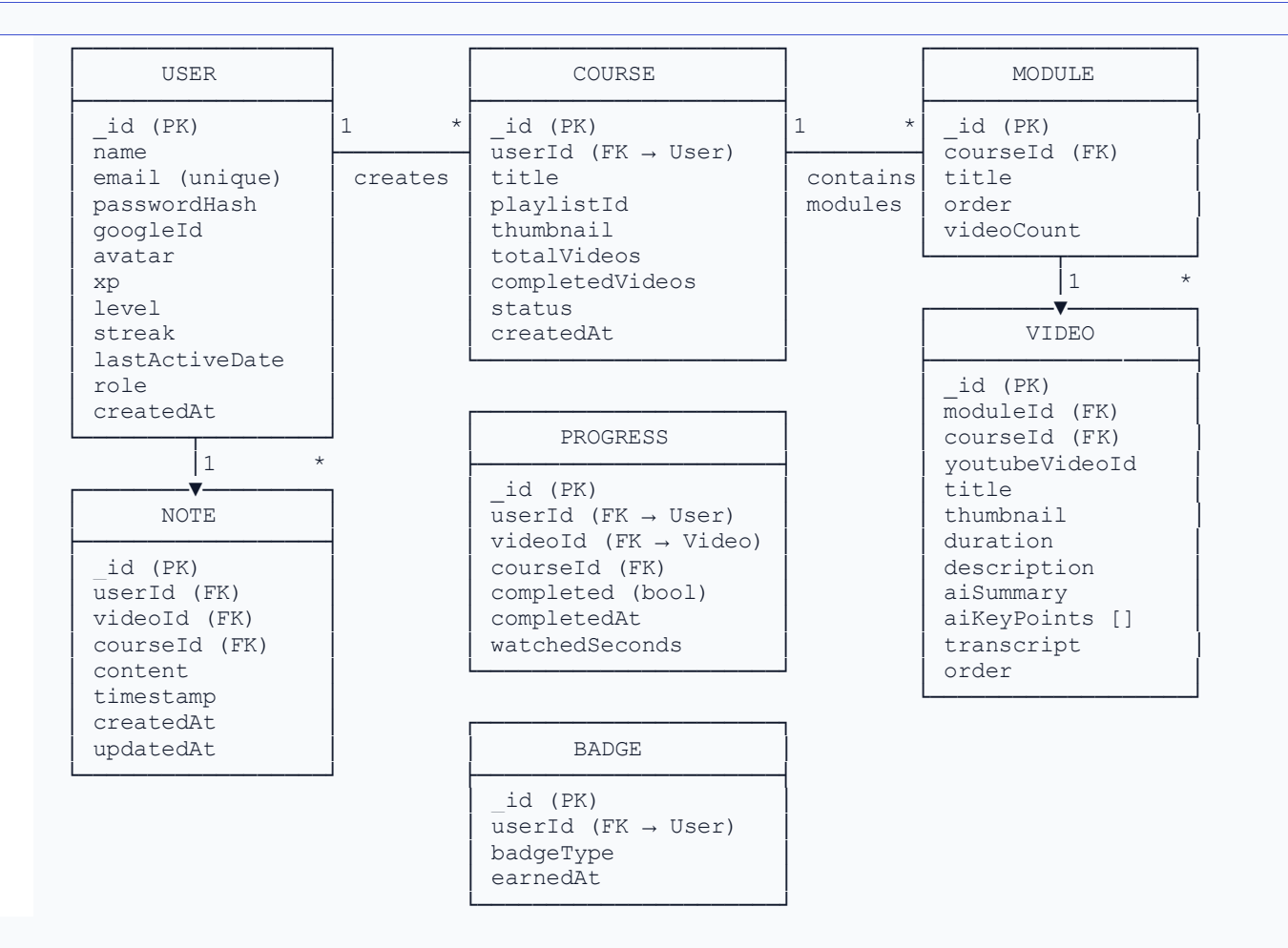
SECTION 4.2

Database Design & Data Modeling

ER Diagram · Logical Schema · Physical Schema

4.2.1 ER Diagram (Entity-Relationship)

Note: The ERD below uses a structured textual representation. Render this in Lucidchart, draw.io, or dbdiagram.io for the final submission.



4.2.2 Logical & Physical Schema

Collection: users

```
{
  _id:           ObjectId,           // Primary Key (auto-generated)
  name:          String,             // Display name – required
  email:         String,             // Unique – indexed
  passwordHash:  String,             // bcrypt hash – null if Google OAuth
  googleId:      String,             // Null if email/password auth
  avatar:        String,             // URL to avatar image
  role:          String,             // 'learner' | 'admin' – default: 'learner'
  xp:            Number,             // Total XP points – default: 0
  level:         Number,             // Computed from XP thresholds – default: 1
  streak:        Number,             // Consecutive days active – default: 0
  lastActiveDate: Date,              // For streak calculation
  resetToken:    String,             // Hashed password reset token
  resetTokenExp: Date,              // Expiry for reset token
  createdAt:     Date,               // Auto timestamps
  updatedAt:     Date
}
// Indexes: email (unique), googleId (sparse unique), xp (desc – for leaderboard)
```

Collection: courses

```
{
  _id:           ObjectId,           // Primary Key
  userId:        ObjectId,           // FK → users._id – required – indexed
  title:         String,             // Course name (from YouTube or user-renamed)
  playlistId:    String,             // YouTube playlist ID – indexed
  thumbnail:     String,             // Playlist thumbnail URL
  channelName:   String,             // YouTube channel name
  totalVideos:   Number,             // Total video count in playlist
  completedVideos: Number,           // Derived from Progress collection
  status:        String,             // 'processing' | 'ready' | 'failed'
  aiJobId:       String,             // Bull/Redis job ID for tracking
  modules:       [ObjectId],         // Ordered refs to Module documents
  createdAt:     Date,
  updatedAt:     Date
}
// Indexes: userId, playlistId+userId (compound unique – prevents duplicates)
```

Collection: modules

```
{
  _id:           ObjectId,           // Primary Key
  courseId:      ObjectId,           // FK → courses._id – indexed
  userId:        ObjectId,           // Denormalized for query efficiency
  title:         String,             // AI-generated module name
  order:         Number,             // Sort order within course
  videos:        [ObjectId],         // Ordered refs to Video documents
  createdAt:     Date
}
```

Collection: videos

```
{
  _id:           ObjectId,           // Primary Key
  moduleId:      ObjectId,           // FK → modules._id – indexed
  courseId:      ObjectId,           // Denormalized FK – indexed
  userId:        ObjectId,           // Denormalized for ownership queries
  youtubeVideoId: String,            // YouTube video ID (e.g., 'dQw4w9WgXcQ')
  title:         String,
  thumbnail:     String,
  duration:      Number,             // Duration in seconds
  description:   String,             // YouTube description (truncated)
  aiSummary:     String,             // LLM-generated summary paragraph
  aiKeyPoints:   [String],           // LLM-generated bullet key points
  transcript:    String,             // Raw transcript (may be omitted after processing)
  order:         Number,             // Position within module
  createdAt:     Date
}
// Indexes: courseId, moduleId, youtubeVideoId+courseId (compound unique)
```

Collection: progress

```
{
  _id:           ObjectId,           // Primary Key
  userId:        ObjectId,           // FK → users._id – indexed
  videoId:       ObjectId,           // FK → videos._id – indexed
  courseId:      ObjectId,           // Denormalized – indexed
  completed:     Boolean,            // Default: false
  completedAt:   Date,              // Null until marked complete
  watchedSeconds: Number,           // Track partial progress – default: 0
  createdAt:     Date,
  updatedAt:     Date
}
// Indexes: userId+videoId (compound unique), userId+courseId (for dashboard agg)
```

Collection: notes

```
{
  _id:           ObjectId,           // Primary Key
  userId:        ObjectId,           // FK → users._id – indexed
  videoId:       ObjectId,           // FK → videos._id – indexed
  courseId:      ObjectId,           // Denormalized – indexed
  content:       String,             // Note text – required
  timestamp:     Number,             // Video timestamp in seconds (optional)
  createdAt:     Date,
  updatedAt:     Date
}
// Indexes: userId, courseId, userId+courseId (compound – for notes page filter)
```

Collection: badges

```
{
  _id:      ObjectId,
  userId:   ObjectId,           // FK → users._id — indexed
  badgeType: String,           // 'first_course' | 'streak_7' | 'videos_50' ...
  badgeLabel:String,           // Display name
  icon:     String,             // Emoji or icon URL
  earnedAt: Date
}
// Indexes: userId, userId+badgeType (compound unique — prevent duplicate awards)
```

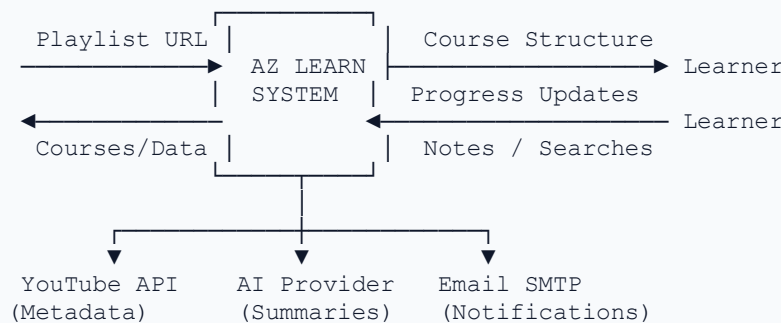
SECTION 4.3

Data Flow & System Behavior

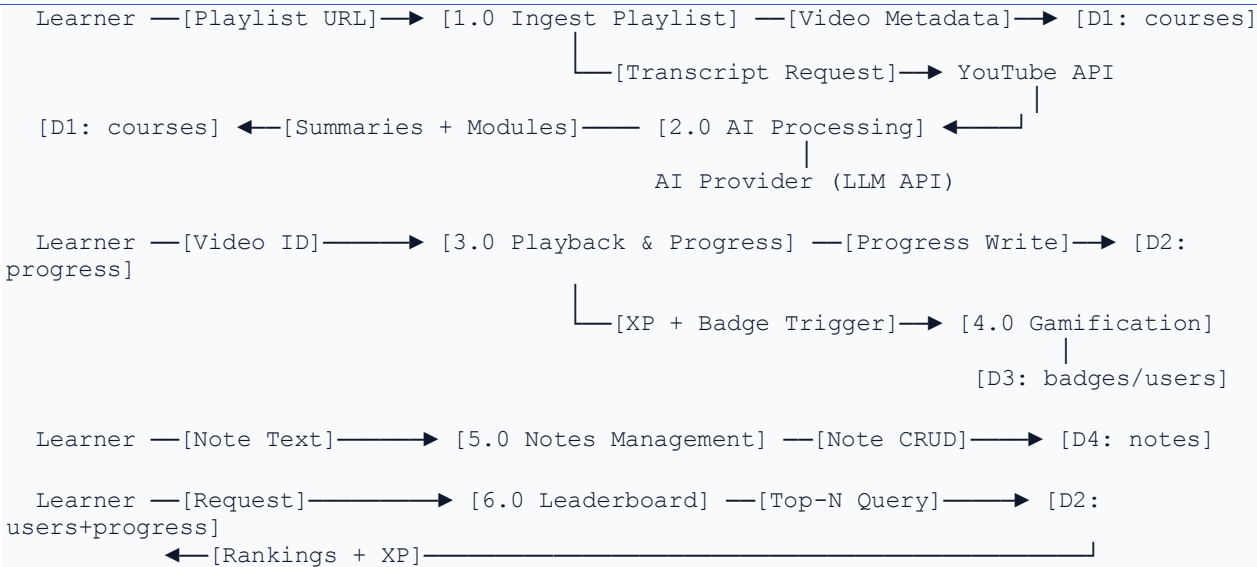
DFD · Sequence · Activity · State · Class Diagrams

4.3.1 Data Flow Diagram (DFD)

Level 0 — Context Diagram

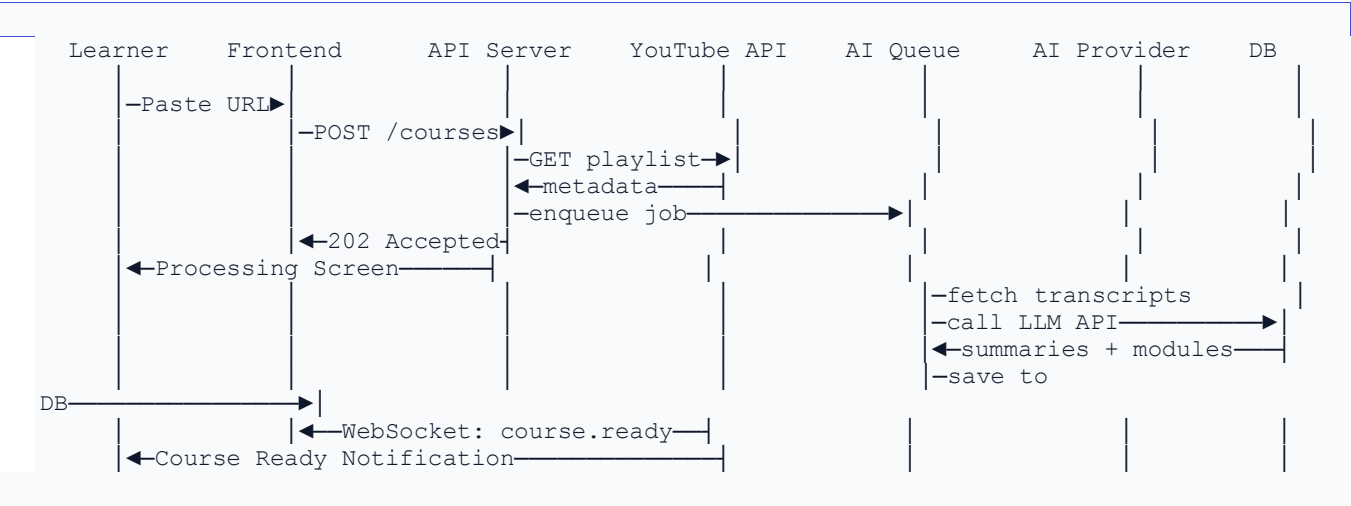


Level 1 — Major Data Flows

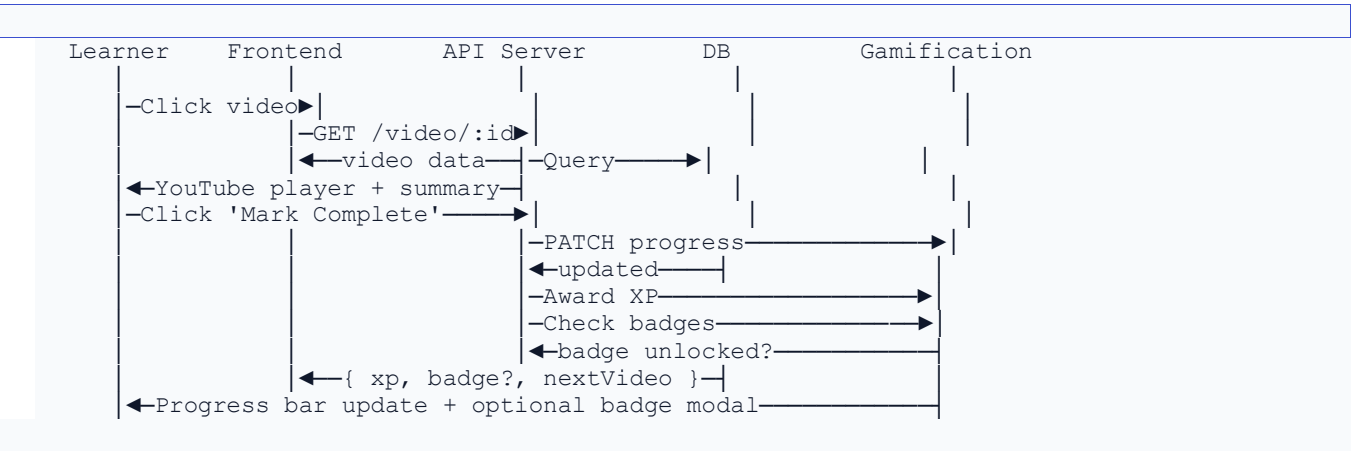


4.3.2 Sequence Diagrams

SD-01: Add Course via YouTube Playlist



SD-02: Watch Video & Mark Complete



4.3.3 Activity Diagram

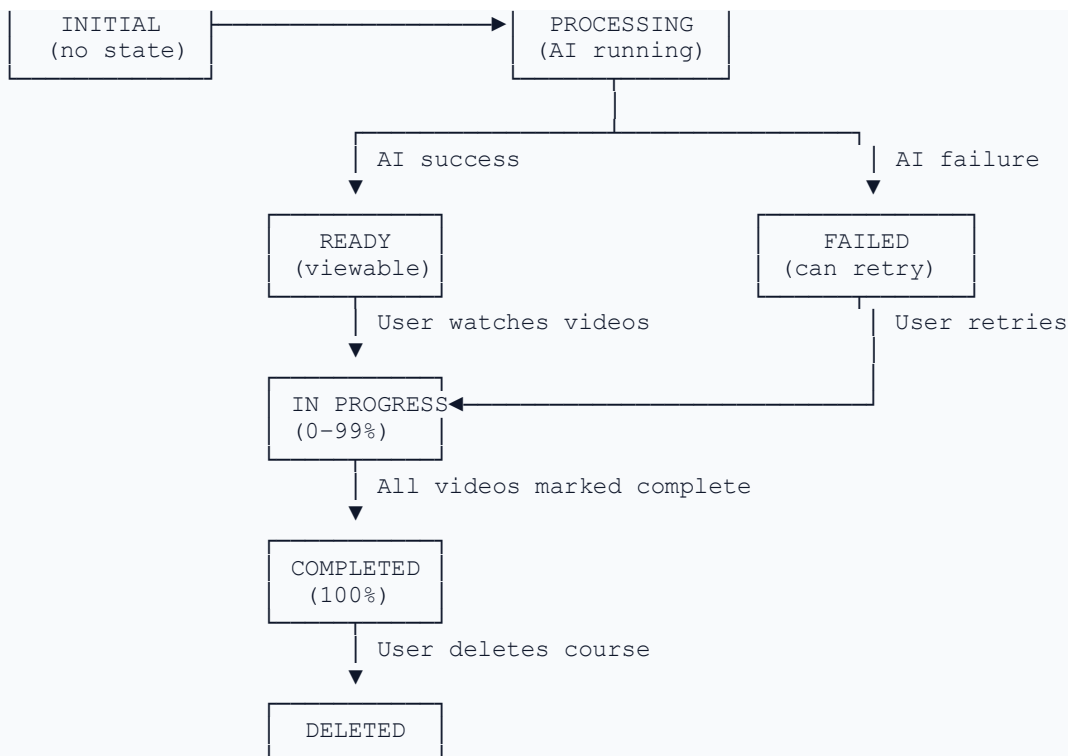
AD-01: Course Creation Workflow



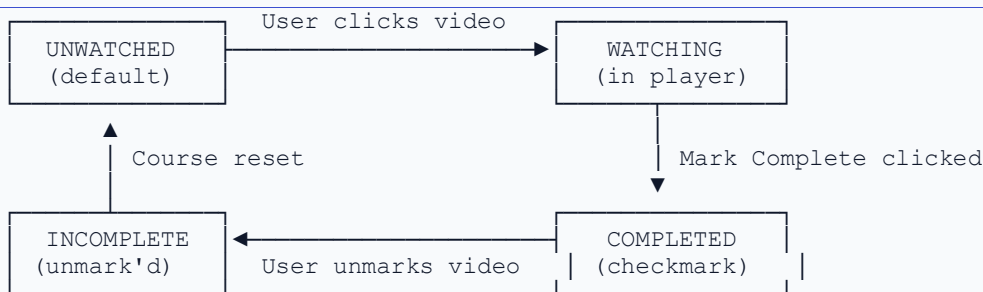
4.3.4 State Diagram

Course Entity State Machine





Video Progress State Machine

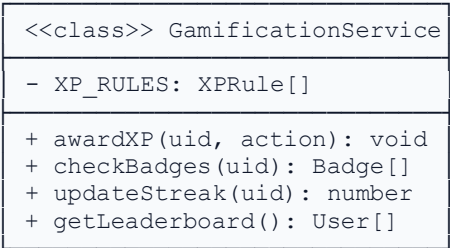
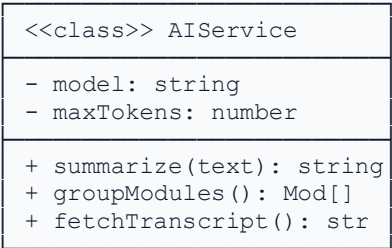
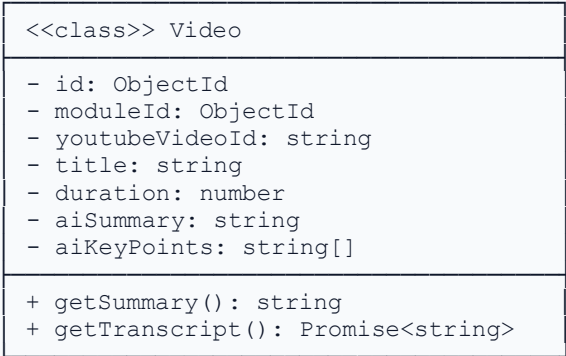
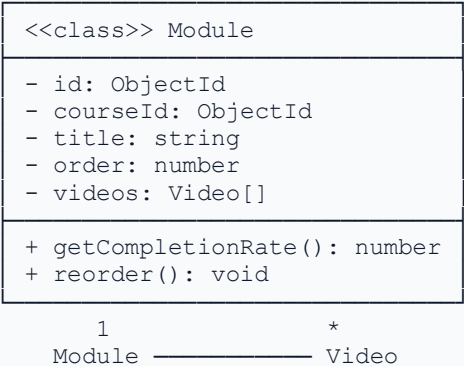
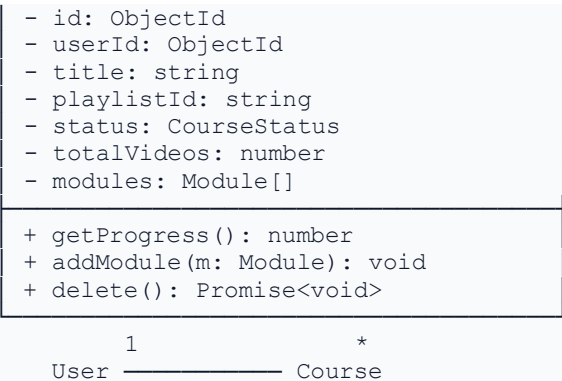
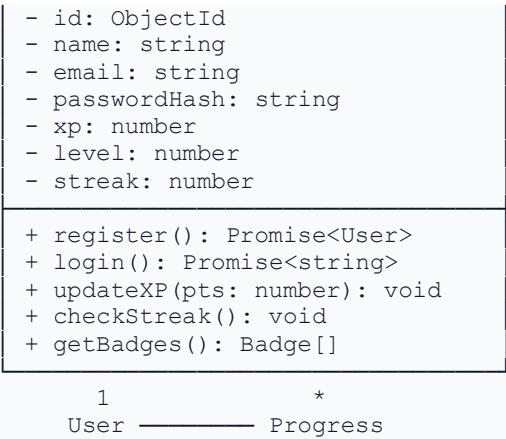


4.3.5 Class Diagram

Note: This is a textual UML Class Diagram. Render it with PlantUML or draw.io for the final submission.

`<<class>> User`

`<<class>> Course`



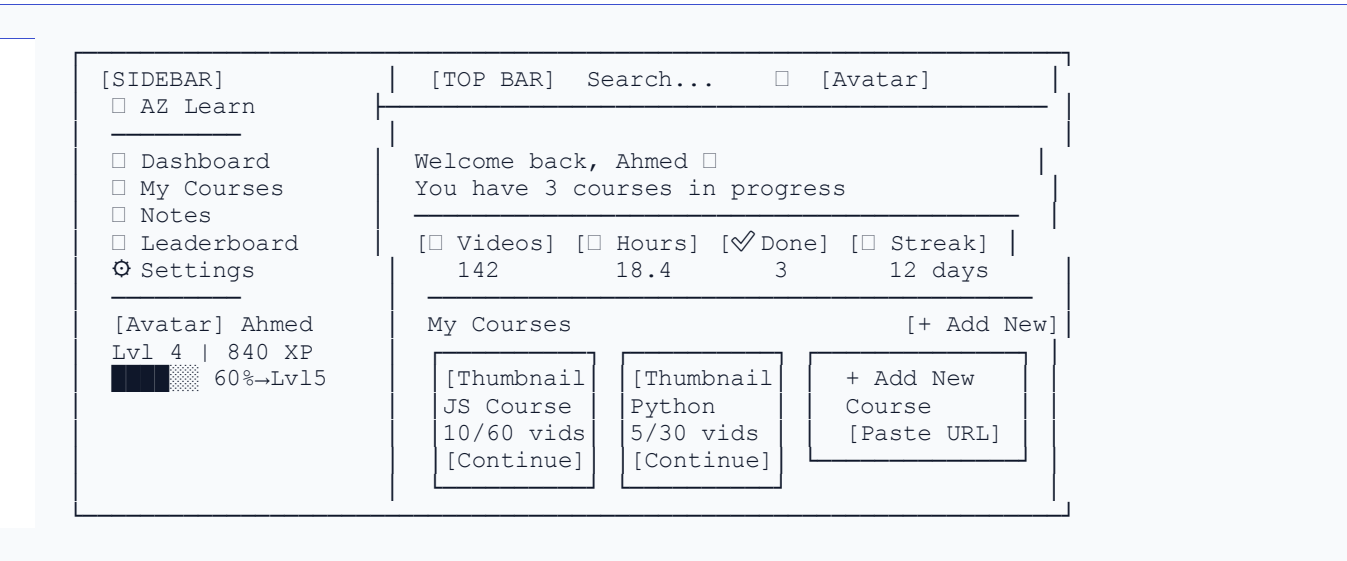
SECTION 4.4

UI/UX Design & Prototyping

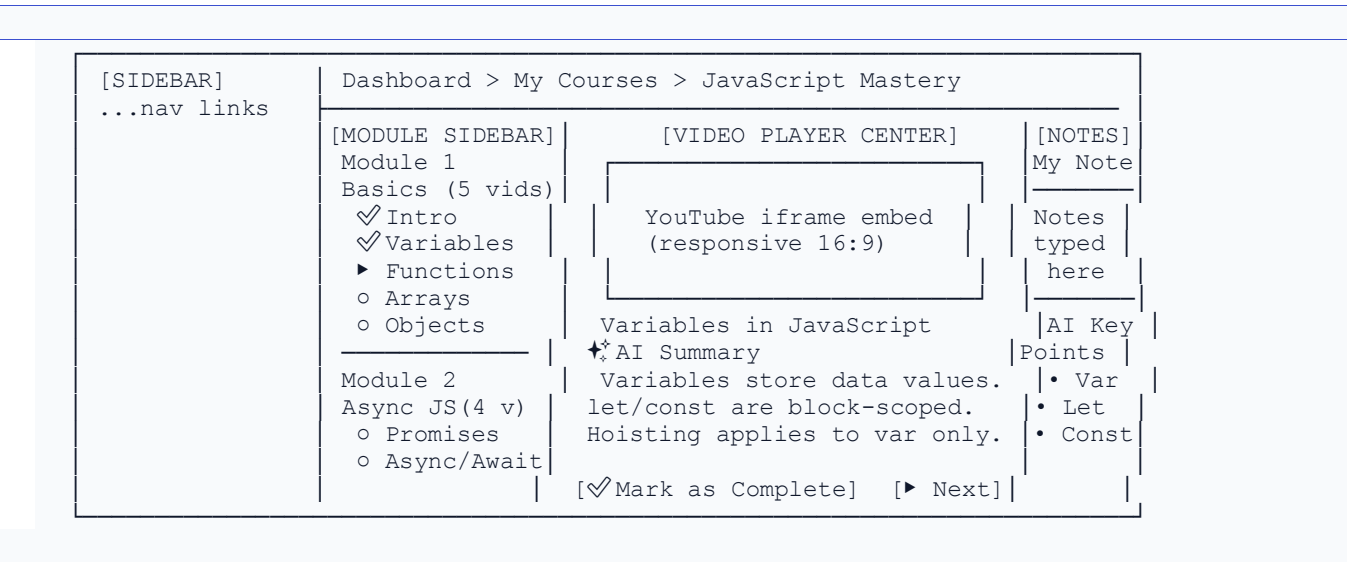
Wireframes · Design System · Accessibility

4.4.1 Wireframes & Screen Descriptions

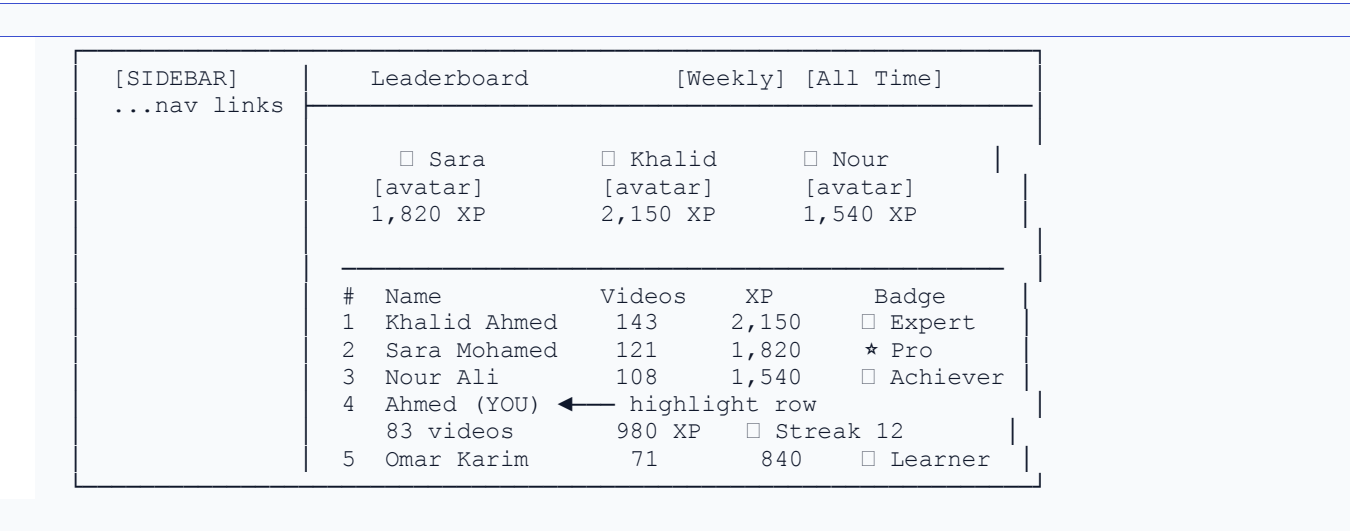
Screen 1 — Dashboard (Main Hub)



Screen 2 — Course Detail Page



Screen 3 — Leaderboard Page



4.4.2 UI/UX Design Guidelines

Color System

Token	Hex Value	Usage	Context
Primary Brand	#1E3A8A	Deep Blue	Sidebar background, main headings, primary actions
Brand Mid	#3B4FCF	Blue-Purple	Buttons, active states, progress fills, links
Accent Purple	#6C63FF	Vibrant Purple	AI feature highlights, badges, gradient end point
Teal	#0D9488	Teal	Success states, section headings, positive indicators
Surface / Card	#FFFFFF	White	Card backgrounds, modal backgrounds, input fields
Page Background	#F1F5F9	Slate 100	Page/layout background in light mode
Dark Background	#0F172A	Slate 950	Dark mode page background, code blocks
Body Text	#374151	Gray 700	All paragraph and body text

Muted Text	#6B7280	Gray 500	Placeholders, timestamps, secondary labels
Border	#CBD5E1	Slate 300	Card borders, dividers, input outlines
Success	#16A34A	Green 600	Completed status, positive badges, streak active
Warning	#D97706	Amber 600	Medium-risk indicators, processing states
Danger	#DC2626	Red 600	Error messages, delete actions, high-risk warnings

Typography System

Element	Font	Size	Weight	Usage
Hero Heading	Plus Jakarta Sans	48–64px	800 ExtraBold	Landing page hero, onboarding screen titles
Page Title (H1)	Plus Jakarta Sans	32–36px	700 Bold	Dashboard section titles, course names
Section Heading (H2)	Inter	24–28px	600 SemiBold	Card titles, modal headings, tab labels
Sub-heading (H3)	Inter	18–20px	600 SemiBold	Module titles, sidebar group labels, form sections
Body Text	Inter	14–16px	400 Regular	Paragraph text, descriptions, notes content
Body Small	Inter	13px	400 Regular	Secondary info, timestamps, meta labels
Caption / Label	Inter	12px	500 Medium	Table headers, badge labels, tooltip text
Code / Monospace	JetBrains Mono	13px	400 Regular	Code snippets, transcript previews

Spacing & Sizing System (8px Grid)

- Base Unit: 8px — all spacing values are multiples of 8 (8, 16, 24, 32, 48, 64, 96px).
- Border Radius: 4px (inputs), 8px (buttons), 12px (cards), 16px (modals), 24px (floating elements).
- Card Shadow: 0 4px 24px rgba(0,0,0,0.08) — light, elevated card feel without heaviness.
- Sidebar Width: 240px expanded / 72px collapsed — transition: 200ms ease-in-out.
- Content Max Width: 1280px centered — never stretches to full viewport on wide screens.

Accessibility Guidelines (WCAG 2.1 AA)

- **Color Contrast:** All text on backgrounds must meet a minimum contrast ratio of 4.5:1 for normal text and 3:1 for large text. Use the WebAIM Contrast Checker to verify.
- **Keyboard Navigation:** Every interactive element must be reachable and operable via keyboard only. Tab order must follow visual reading order left-to-right, top-to-bottom.
- **Focus Indicators:** All focusable elements must display a visible focus ring: outline: 2px solid #6C63FF; outline-offset: 2px.
- **ARIA Labels:** All icon-only buttons must have aria-label. Dynamic content (progress bars, live scores) must use aria-live='polite'.
- **Skip Navigation:** A 'Skip to main content' link must be the first focusable element on every page for screen reader users.
- **Alt Text:** All images, thumbnails, and badge icons must have descriptive alt text. Decorative images use alt="".

SECTION 4.5

System Deployment & Integration

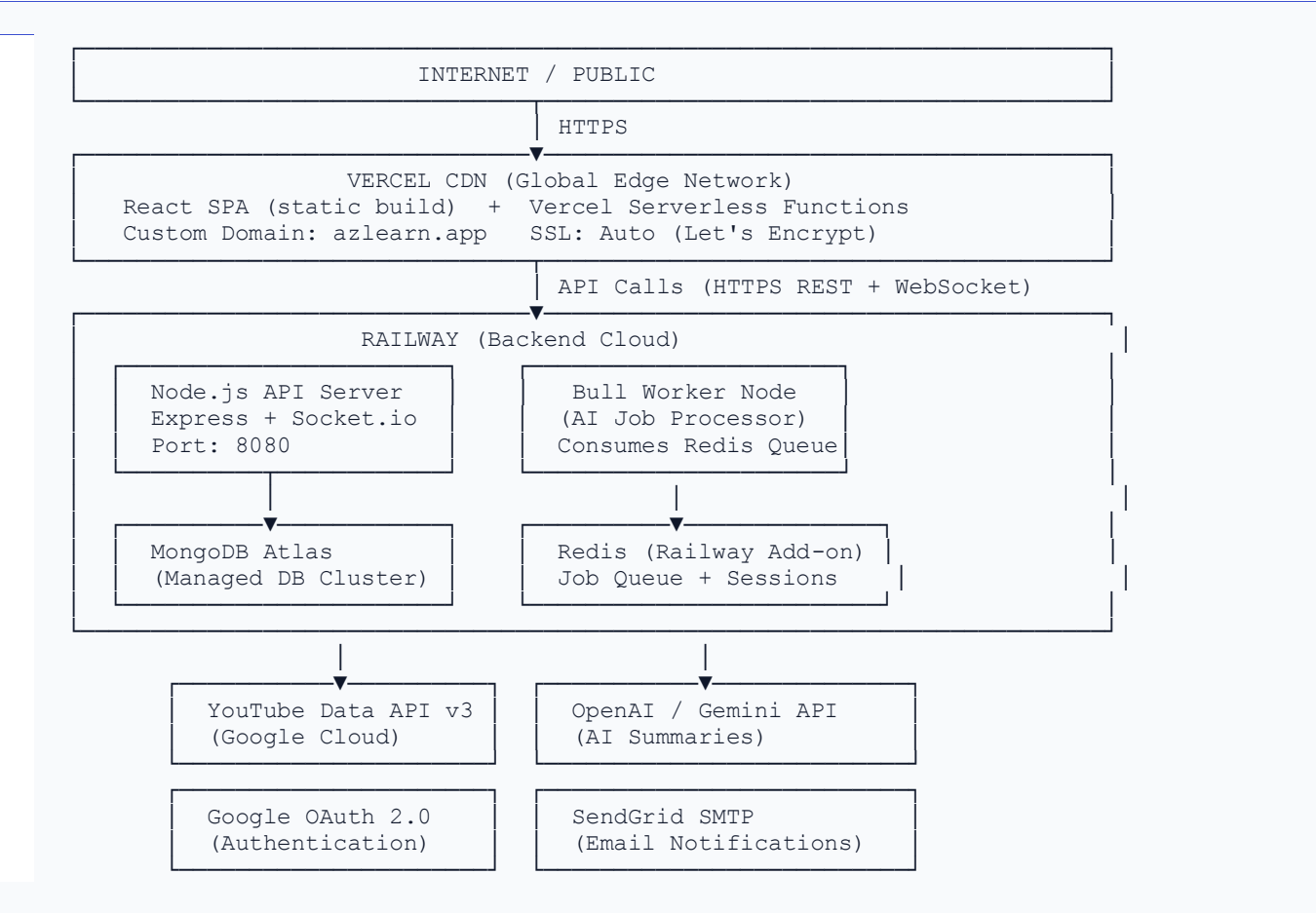
Tech Stack · Deployment · Component Diagrams

4.5.1 Technology Stack

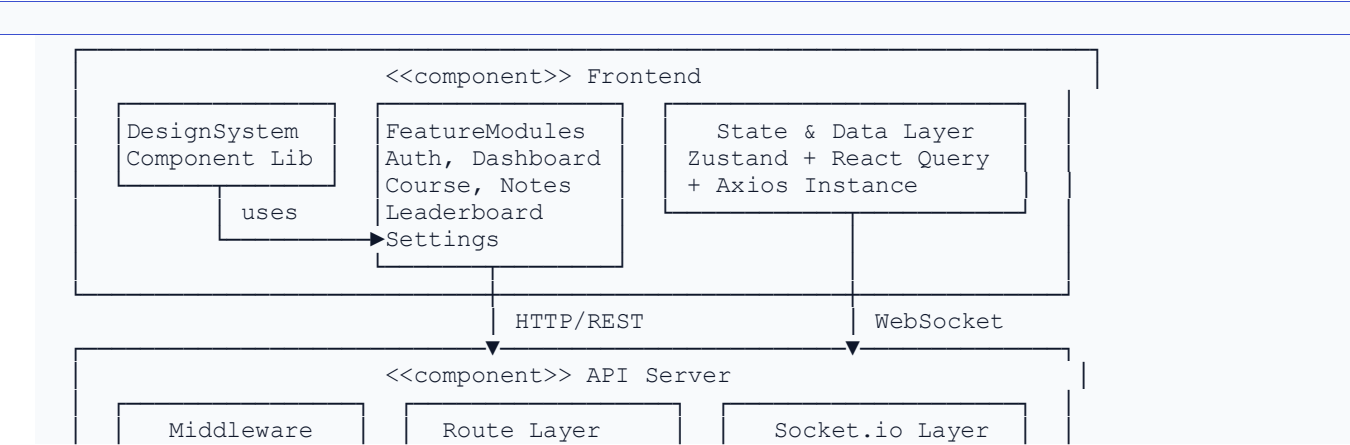
Layer	Technology	Version / Detail	Justification
Frontend	React + TypeScript	React 18, TS 5.x, Vite build tool	Component-based, strong typing, fast HMR dev experience
Frontend	Tailwind CSS	v3.x, JIT mode	Utility-first, design token support, no CSS bloat
Frontend	Zustand	v4.x	Lightweight global state — no boilerplate vs Redux
Frontend	React Query	TanStack Query v5	Server state management, caching, background refetch
Frontend	React Hook Form + Zod	Latest	Performant forms with type-safe schema validation
Frontend	Framer Motion	v11.x	Production-grade animations: page transitions, badge reveal
Backend	Node.js + Express	Node 20 LTS, Express 4.x	Non-blocking I/O — ideal for high-concurrency API + WebSocket
Backend	TypeScript	v5.x with strict mode	End-to-end type safety; shared types with frontend monorepo
Backend	Socket.io	v4.x	Bidirectional WebSocket — AI job completion + leaderboard updates
Backend	Bull + Redis	Bull v4, Redis 7.x	Reliable async job queue for AI processing — retries, delays, progress
Database	MongoDB Atlas	v7.x, Mongoose ODM	Flexible schema for evolving AI content; horizontal scaling; Atlas free tier
Auth	JWT + bcrypt	jsonwebtoken, bcrypt cost 12	Stateless auth tokens; httpOnly cookies; password hashing
Auth	Google OAuth 2.0	Passport.js google-oauth20	One-click login; reduces friction for new users

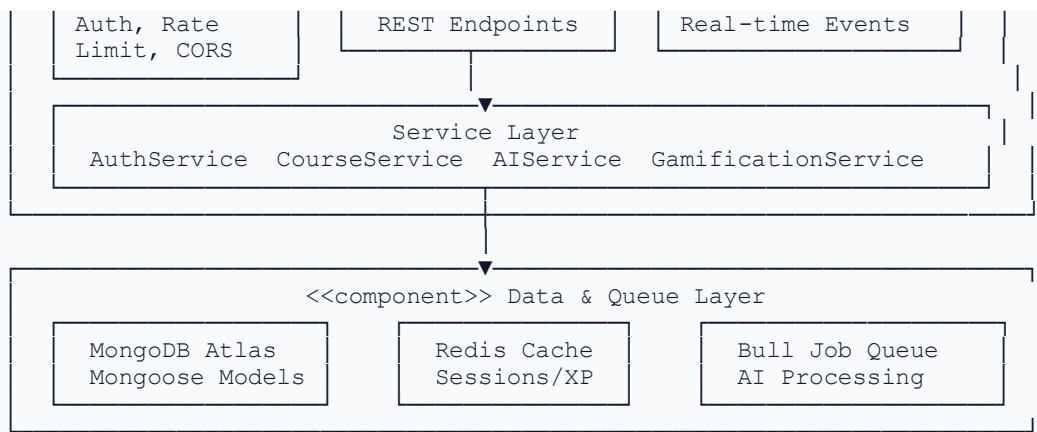
AI	OpenAI API / Gemini	GPT-4o-mini / Gemini 1.5 Flash	Best cost-accuracy balance for summarization at scale
External	YouTube Data API v3	Google Cloud, 10K quota/day	Official playlist and video metadata source
External	youtube-transcript	npm package	Fetch auto-generated + manual captions for AI input
DevOps	Vercel (Frontend)	Automatic CI from GitHub	Zero-config React deployment; instant preview URLs per PR
DevOps	Railway / Render (Backend)	Managed Node.js hosting	Auto-deploy from GitHub; managed Redis and MongoDB add-ons
DevOps	GitHub Actions	CI/CD workflows	Lint → Test → Build → Deploy pipeline on every push to main
Monitoring	Sentry + UptimeRobot	Error tracking + uptime	Real-time error alerts; 99.5% uptime SLA monitoring

4.5.2 Deployment Diagram



4.5.3 Component Diagram





SECTION 4.6

Additional Deliverables

API Docs · Testing Plan · Deployment Strategy

4.6.1 API Documentation

Base URL: <https://api.azlearn.app/v1> | Auth: Bearer JWT token in Authorization header

Authentication Endpoints

Method	Endpoint	Body / Params	Response	Description
POST	/auth/register	{ name, email, password }	201: { user, token }	Register new user account
POST	/auth/login	{ email, password }	200: { user, token }	Login with credentials
GET	/auth/google	—	302: OAuth redirect	Initiate Google OAuth flow
GET	/auth/google/callback	?code=...	302: /dashboard	Google OAuth callback handler
POST	/auth/forgot-password	{ email }	200: { message }	Send password reset email
POST	/auth/reset-password	{ token, newPassword }	200: { message }	Reset password with valid token
POST	/auth/logout	—	200: { message }	Invalidate session / clear cookie

Course Endpoints

Method	Endpoint	Body / Params	Response	Description
GET	/courses	—	200: Course[]	Get all courses for authenticated user
POST	/courses	{ playlistUrl }	202: { courseId, jobId }	Create course from YouTube playlist URL — triggers AI job
GET	/courses/:id	—	200: CourseDetail	Get full course detail with modules and videos
PATCH	/courses/:id	{ title }	200: Course	Rename a course

DELETE	/courses/:id	—	204: No Content	Delete course and all associated progress/notes
GET	/courses/:id/progress	—	200: { pct, completed, total }	Get completion percentage for a course

Video & Progress Endpoints

Method	Endpoint	Body	Response	Description
GET	/videos/:id	—	200: VideoDetail	Get video detail with AI summary and key points
POST	/videos/:id/complete	—	200: { xp, badge? }	Mark video as complete — triggers XP award
DELETE	/videos/:id/complete	—	200: OK	Unmark a video as complete
PATCH	/videos/:id/progress	{ watchedSeconds }	200: Progress	Update partial watch progress in seconds
GET	/notes	?courseId&keyword&page	200: Note[]	Get all notes with optional filters
POST	/notes	{ videoId, courseId, content }	201: Note	Create a new note linked to a video
PATCH	/notes/:id	{ content }	200: Note	Update note content (auto-save)
DELETE	/notes/:id	—	204: No Content	Delete a specific note
GET	/leaderboard	?period=weekly alltime	200: LeaderboardEntry[]	Get ranked leaderboard — cached 60s in Redis
GET	/users/me	—	200: UserProfile	Get current user profile, XP, level, badges, streak

WebSocket Events

Direction	Event Name	Payload	Description
Server → Client	course.ready	<code>{ courseId, moduleCount }</code>	AI processing complete — course is ready to view
Server → Client	course.failed	<code>{ courseId, error }</code>	AI job failed — show retry option to user
Server → Client	leaderboard.update	<code>{ userId, xp, rank }</code>	A user's XP changed — update leaderboard in real-time
Server → Client	badge.unlocked	<code>{ badge: { type, label, icon } }</code>	User unlocked a new badge — trigger reveal animation
Client → Server	join.course	<code>{ courseId }</code>	Subscribe to job notifications for a specific course
Client → Server	leave.course	<code>{ courseId }</code>	Unsubscribe from course notifications

4.6.2 Testing & Validation Plan

Unit Testing (Jest)

- **AuthService:** Test password hashing, JWT generation, token expiry, Google token verification.
- **CourseService:** Test duplicate detection, progress calculation, module assembly from AI output.
- **AIService:** Mock LLM API responses — test prompt construction, response parsing, fallback on failure.
- **GamificationService:** Test XP award rules, badge unlock conditions, streak calculation logic.
- **Target Coverage:** > 80% line coverage on all service files.

Integration Testing (Supertest)

- **Auth flow:** POST /register → POST /login → GET /users/me — verify JWT is issued and identity returned.
- **Course lifecycle:** POST /courses → GET /courses/:id → PATCH → DELETE — verify all state transitions.
- **Progress lifecycle:** POST /videos/:id/complete → GET /courses/:id/progress — verify percentage calculation.
- **Notes CRUD:** POST → GET (with filter) → PATCH → DELETE — verify filtering and auto-save.

Frontend Component Testing (React Testing Library)

- **ProgressBar component:** Renders correct width% given completed/total props.
- **CourseCard component:** Shows thumbnail, title, video count, and progress percentage.
- **Add Course Wizard:** Step 1 → 2 → 3 transitions; URL validation error message shown on invalid input.
- **PrivateRoute:** Redirects to /login if no valid JWT is present in cookie/context.

End-to-End Testing (Playwright)

- **Full user journey:** Register → Add Course → Watch video → Mark Complete → View updated progress bar.
- **Leaderboard update:** Two sessions open — one marks video complete — other sees updated score.
- **Form validation:** Attempt login with wrong password → error message rendered.

Performance Testing

- **Tool:** k6 load testing — simulate 100 concurrent users hitting GET /courses and POST /videos/:id/complete simultaneously.

- **Pass Criteria:** P95 response time < 300ms; no error rate above 0.5%; server memory stable under sustained load.

User Acceptance Testing (UAT)

ID	Test Scenario	Expected Result	Pass Criteria
UAT-1	New user registers, pastes a 20-video playlist URL, and waits for processing	Course appears in dashboard with modules and AI summaries	Processing completes in < 60s; all videos present; summaries non-empty
UAT-2	User watches 3 videos and marks them complete	Dashboard progress bar updates to 15% (3/20)	Percentage is mathematically correct; persists on page refresh
UAT-3	User writes a note on video 5, navigates away, returns	Note is still present and linked to correct video	Note auto-saved within 1 second of last keystroke; survives navigation
UAT-4	Two users complete videos — leaderboard updates live	Both users see updated rankings within 2 seconds	WebSocket event received; UI updates without page refresh
UAT-5	User attempts to add the same playlist twice	System rejects with 'Course already exists' message	No duplicate created in database; informative error shown

4.6.3 Deployment Strategy

Environments

Environment	Purpose	Trigger	URL Pattern
Development	Local coding — hot reload, local MongoDB	Manual — <i>npm run dev</i>	localhost:5173 (FE) / localhost:8080 (BE)
Preview	PR review — auto-deployed per branch	GitHub PR opened	pr-{id}.azlearn.vercel.app (FE) / Railway preview env
Staging	Pre-production — mirrors production config	Push to dev branch	staging.azlearn.app
Production	Live platform — all users	Push to main (after staging approval)	azlearn.app / api.azlearn.app

CI/CD Pipeline (GitHub Actions)

```
Developer pushes to feature branch
↓
[GitHub Actions: PR Pipeline]
1. Checkout code
2. Install dependencies (npm ci)
3. TypeScript compile check (tsc --noEmit)
4. ESLint + Prettier check
5. Run unit tests (jest --coverage) — fail if < 80%
6. Run integration tests (supertest)
7. Build frontend (vite build)
↓
  ▼ All steps pass
[Vercel: Auto-deploy Preview URL] ← Share with reviewer
  ▼ PR merged to dev
[Deploy to Staging: Railway + Vercel staging]
  ▼ Manual approval in GitHub
[Deploy to Production: Railway + Vercel]
  ▼
[Sentry: verify no spike in errors]
[UptimeRobot: confirm endpoint health checks pass]
```

Scaling Considerations

- **Stateless API Servers:** JWT auth and session data are stored in Redis — no server-side sessions — allowing multiple API instances behind a load balancer without sticky sessions.
- **MongoDB Atlas Auto-scaling:** Atlas M10+ clusters auto-scale storage. Indexes on high-query fields (userId, courseId, xp) ensure query time stays under 100ms at scale.
- **Bull Job Workers:** AI processing workers run as separate Node.js processes. Adding more workers horizontally scales throughput — multiple courses can be processed in parallel.
- **CDN for Static Assets:** All frontend assets are served from Vercel's global CDN with cache headers — static JS/CSS bundles are cached at the edge closest to each user.
- **Redis Caching Strategy:** Leaderboard top-100 cached with 60-second TTL. Course metadata cached with 5-minute TTL. Invalidated on mutation events.

AZ Learn — From A to Z

End of Section 4 — System Analysis & Design